

C# - Phần 9.8: Event cơ bản

Từ Delegate và Multicast Delegate đến cơ chế thông báo bằng Event

Ý quan trọng nhất: Event là cơ chế thông báo dựa trên Delegate. Một object có thể phát thông báo khi một việc xảy ra, còn các method khác có thể đăng ký để được gọi khi sự kiện đó xảy ra.

1. Từ Multicast Delegate sang Event

```
Action<string> notification = NotificationService.SendEmail;

notification += NotificationService.WriteLog;

notification("Order completed");
```

Multicast Delegate cho phép một biến Delegate giữ nhiều method. Nhưng nếu expose Delegate trực tiếp ra ngoài, code khác có thể gán lại toàn bộ danh sách hoặc tự gọi Delegate. Với Event, bên ngoài chủ yếu chỉ được đăng ký/hủy đăng ký, còn việc phát Event do class sở hữu Event quyết định.

2. Ví dụ Event đầu tiên

```
public class OrderService
{
    public event Action<string>? OrderCompleted;
}
```

- OrderCompleted là một Event.
- Action<string> nghĩa là Event dùng Delegate nhận string và trả void.
- Dấu ? vì lúc mới tạo object có thể chưa có method nào đăng ký Event.

3. Phát Event

```
public void CompleteOrder(string orderId)
{
    Console.WriteLine(
        $"Order {orderId} has been completed.");

    OrderCompleted?.Invoke(orderId);
}
```

Dòng OrderCompleted?.Invoke(orderId) nghĩa là: nếu Event có người đăng ký thì gọi các method đã đăng ký và truyền orderId cho chúng.

4. Invoke() là gì?

Với Delegate, hai cách sau gần tương đương:

```
notification("Hello");

notification.Invoke("Hello");
```

Với Event, thường gặp:

```
OrderCompleted?.Invoke(orderId);
```

Invoke() nghĩa là gọi Delegate/Event. Toán tử ?. chỉ gọi nếu Event không phải null.

5. Vì sao cần ?.Invoke()?

Nếu chưa có subscriber nào đăng ký thì Event có thể là null. ?.Invoke() giúp bỏ qua an toàn thay vì cố gọi một giá trị null.

Có thể hình dung:

- Có subscriber -> gọi các method đã đăng ký.
- Không có subscriber -> bỏ qua.

6. Tạo các method nhận thông báo

```
public static class NotificationService
{
    public static void SendEmail(string orderId)
    {
        Console.WriteLine(
            $"Email sent for order {orderId}.");
    }

    public static void WriteLog(string orderId)
    {
        Console.WriteLine(
            $"Order {orderId} was written to log.");
    }
}
```

Hai method đều có chữ ký string -> void nên phù hợp với Action<string> của Event.

7. Đăng ký Event bằng +=

```
OrderService orderService =
    new OrderService();

orderService.OrderCompleted +=
    NotificationService.SendEmail;

orderService.OrderCompleted +=
    NotificationService.WriteLog;
```

Lúc này SendEmail và WriteLog mới chỉ đăng ký. Chúng chưa chạy cho đến khi OrderService phát Event.

8. Khi nào các method chạy?

```
orderService.CompleteOrder("DH01");
```

Bên trong CompleteOrder(), OrderCompleted?.Invoke(orderId) được thực hiện.

- CompleteOrder("DH01")

- Order hoàn thành
- OrderCompleted?.Invoke("DH01")
- SendEmail("DH01")
- WriteLog("DH01")

9. Hủy đăng ký bằng -=

```
orderService.OrderCompleted -=
    NotificationService.SendEmail;
```

Sau đó, khi Event được phát lại thì SendEmail không còn được gọi, còn WriteLog vẫn chạy.

10. Event khác Delegate public bình thường ở đâu?

Nếu expose Delegate public:

```
public Action<string>? OrderCompleted;
```

Code bên ngoài có thể gán lại hoặc tự gọi Delegate.

Nếu dùng Event:

```
public event Action<string>? OrderCompleted;
```

Code bên ngoài chủ yếu chỉ được dùng += và -=. Việc Invoke Event nằm trong class khai báo Event.

```
orderService.OrderCompleted += SendEmail;
orderService.OrderCompleted -= SendEmail;
```

Bên ngoài không được tự phát Event như một Delegate public bình thường.

11. Vì sao điều này hữu ích?

OrderService là nơi biết chính xác khi nào một Order thật sự hoàn thành. Vì vậy OrderService mới có quyền phát Event. Các thành phần khác chỉ đăng ký để được thông báo.

- OrderService -> nơi phát thông báo.
- EmailService / LogService / NotificationService -> nơi đăng ký nhận thông báo.

12. Có thể dùng Custom Delegate với Event

```
public delegate void OrderCompletedHandler(
    string orderId);

public event OrderCompletedHandler?
    OrderCompleted;
```

Không bắt buộc dùng Action<string>. Tuy nhiên ở bài này dùng Action<string> để đơn giản vì bạn đã học Action.

13. Event không phải method

```
public event Action<string>? OrderCompleted;
```

OrderCompleted không phải method như CompleteOrder(). Nó là một Event dựa trên Delegate và giữ danh sách các method đã đăng ký.

14. Event và Multicast Delegate liên hệ như thế nào?

Multicast Delegate là nền tảng cho việc gọi nhiều method. Event dùng Delegate để xây một cơ chế thông báo có kiểm soát hơn.

- Multicast Delegate -> một Delegate có thể có nhiều method.
- Event -> các method đăng ký nhận thông báo bằng += và hủy bằng -=.
- Class sở hữu Event quyết định khi nào gọi Invoke().

15. Publisher và Subscriber - nhận diện sơ bộ

- Publisher -> object phát Event.
- Subscriber -> object/method đăng ký Event.

Trong ví dụ: OrderService là Publisher; SendEmail và WriteLog là các handler của Subscriber. Phần 9.9 sẽ học kỹ hơn.

16. Ví dụ hoàn chỉnh

```
public class OrderService
{
    public event Action<string>? OrderCompleted;

    public void CompleteOrder(string orderId)
    {
        Console.WriteLine(
            $"Order {orderId} has been completed.");

        OrderCompleted?.Invoke(orderId);
    }
}

public static class NotificationService
{
    public static void SendEmail(string orderId)
    {
        Console.WriteLine(
            $"Email sent for order {orderId}.");
    }

    public static void WriteLog(string orderId)
    {
        Console.WriteLine(
            $"Order {orderId} was written to log.");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        OrderService orderService =
```

```

        new OrderService();

        orderService.OrderCompleted +=
            NotificationService.SendEmail;

        orderService.OrderCompleted +=
            NotificationService.WriteLog;

        orderService.CompleteOrder("DH01");

        orderService.OrderCompleted -=
            NotificationService.SendEmail;

        orderService.CompleteOrder("DH02");
    }
}

```

17. Điều cần nhớ ở bài 9.8

- Delegate -> tham chiếu method.
- Multicast Delegate -> có thể chứa nhiều method.
- Event -> cơ chế thông báo dựa trên Delegate.
- Khai báo: `public event Action<string>? OrderCompleted;`
- Đăng ký: `OrderCompleted += Method;`
- Hủy đăng ký: `OrderCompleted -= Method;`
- Phát Event: `OrderCompleted?.Invoke(value);`
- Thông thường chỉ class khai báo Event mới phát/invoke Event đó.

Bài tập 9.8 - Event cơ bản

Tạo project/file:

```

EventBasic/
├── OrderService.cs
├── NotificationService.cs
└── Program.cs

```

OrderService có:

```
public event Action<string>? OrderCompleted;
```

và method:

```
public void CompleteOrder(string orderId)
```

- Kiểm tra orderId null/empty/whitespace.
- Trim.
- In thông báo Order đã hoàn thành.
- Phát OrderCompleted.

NotificationService có:

```

SendEmail(string orderId)
WriteLog(string orderId)
ShowNotification(string orderId)

```

Cả ba trả void.

Trong Program:

- 1. Tạo OrderService.
- 2. Đăng ký SendEmail.
- 3. Đăng ký WriteLog.
- 4. Gọi CompleteOrder("DH01").
- 5. Đăng ký thêm ShowNotification.
- 6. Gọi CompleteOrder("DH02").
- 7. Hủy đăng ký SendEmail.
- 8. Gọi CompleteOrder("DH03").

Câu hỏi cuối bài

- 1. Event được xây dựng dựa trên kiến thức nào?
- 2. += và -= có vai trò gì với Event?
- 3. Vì sao dùng ?.Invoke()?
- 4. Vì sao code bên ngoài không nên được tự phát OrderCompleted?
- 5. Event khác một biến Delegate public bình thường ở điểm quan trọng nào?

Mục tiêu hoàn thành: hiểu được luồng "đăng ký -> một việc xảy ra -> phát Event -> các method đăng ký được gọi".